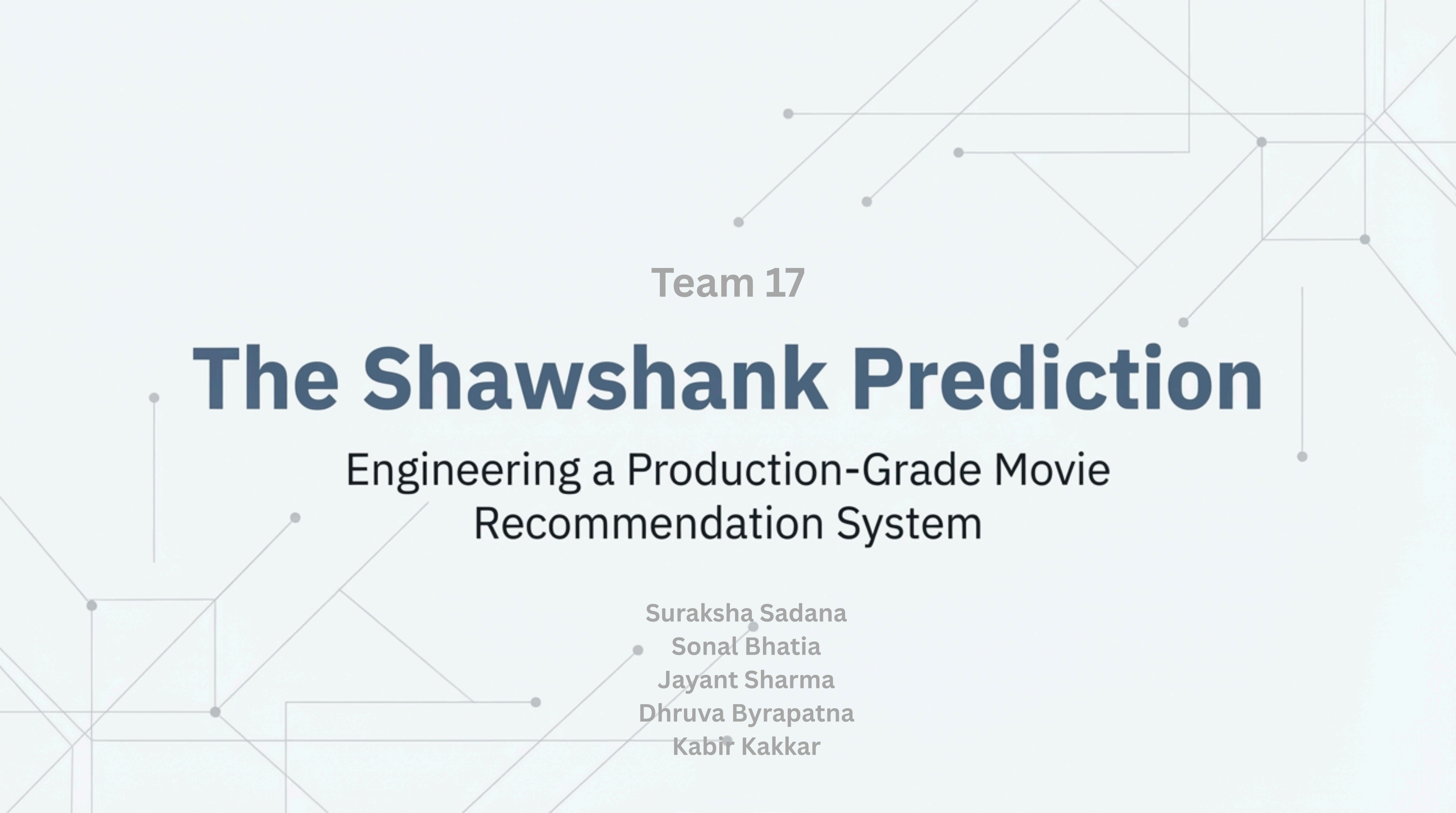




Team 17

The Shawshank Prediction

Engineering a Production-Grade Movie
Recommendation System

Suraksha Sadana

Sonal Bhatia

Jayant Sharma

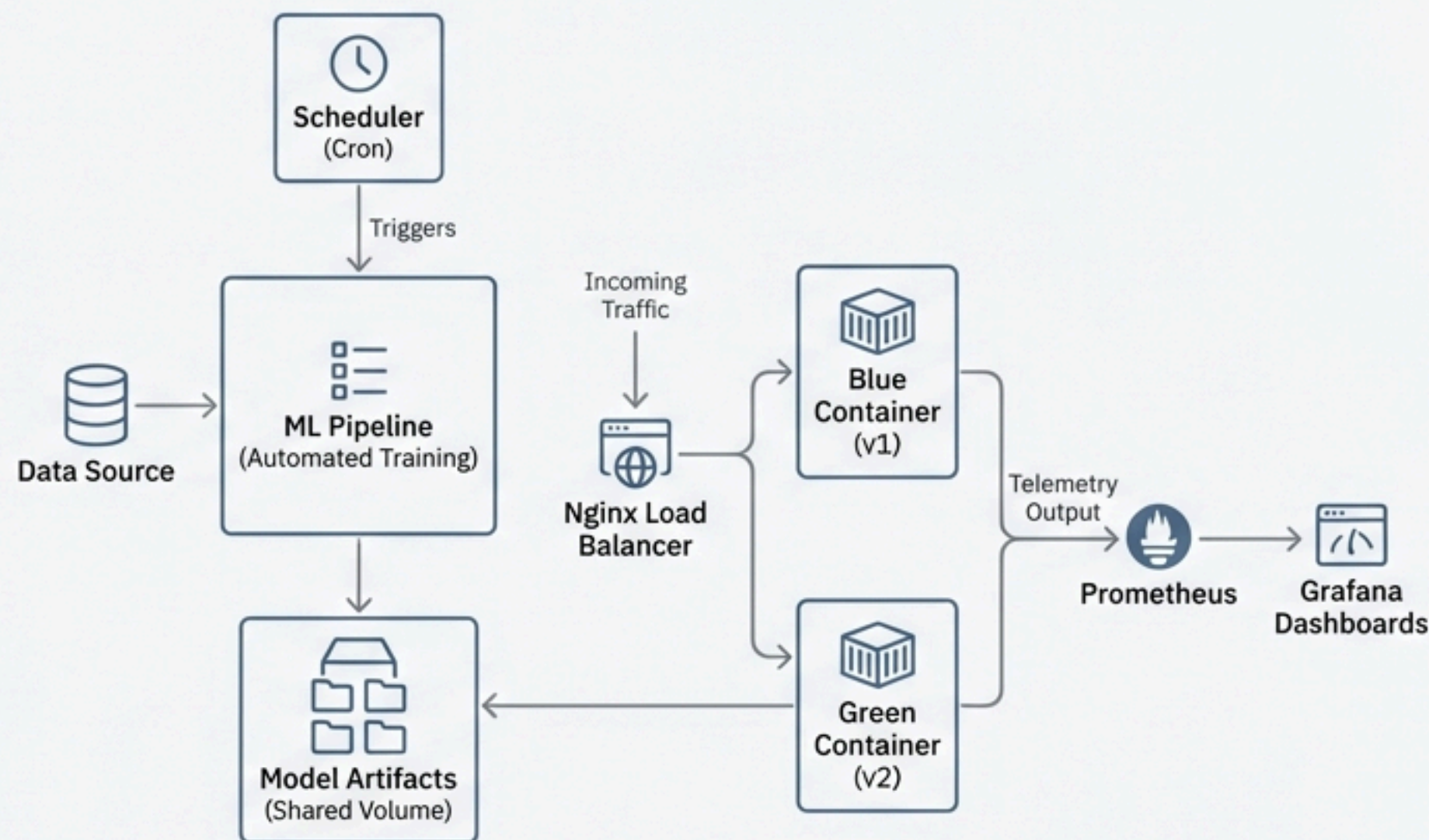
Dhruva Byrapatna

Kabir Kakkar

A recommender is more than a model; it's an end-to-end system.

Our goal was to mirror a real-world ML workflow, focusing on the design decisions that enable reliability, monitoring, and fairness at scale.

- ✓ Personalized SVD Model with Popularity Fallback
- ✓ Automated ML Pipeline with CI/CD
- ✓ Containerized Blue/Green Deployments
- ✓ Comprehensive Monitoring & Drift Detection
- ✓ Fairness Analysis & Feedback Loop Mitigation



The Engine: Choosing an efficient and accurate core model.

The Challenge: Select a model that balances accuracy with the practical constraints of daily retraining.

SVD vs. User-User Collaborative Filtering

Metric	SVD	User-User CF
Accuracy (RMSE/MAE)	Better	Lower
Model Size	3.5 MB	47 MB
Training Time	Faster	Slower
Inference Latency	~18ms	~18ms

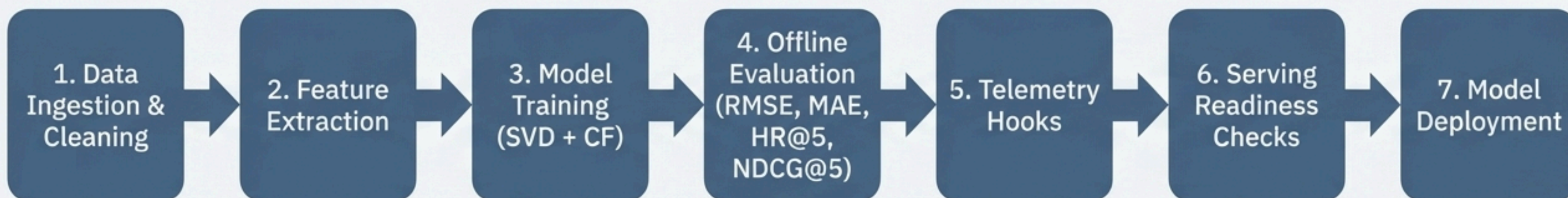
Decision: SVD

It's smaller, faster to train, and more accurate, making it ideal for our daily retraining cadence.

A top-20 popularity model was implemented as a fallback to handle cold-start users and guarantee low latency.

The Factory: Building a repeatable, automated ML pipeline.

Our goal: An end-to-end pipeline executable with a single command.



This modular design ensures each step is testable and that the entire workflow is reproducible.

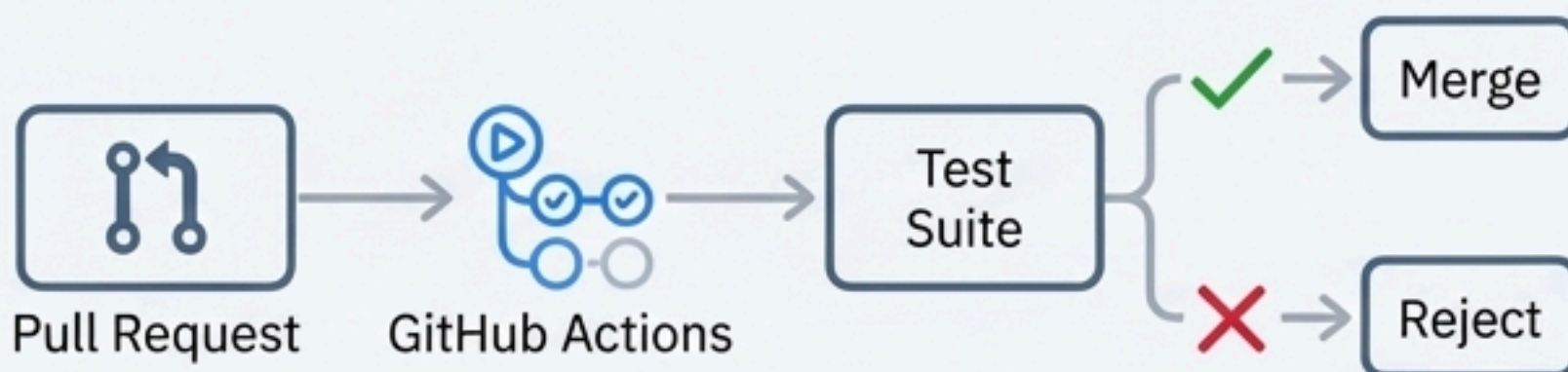
Enforcing Production Discipline with Continuous Integration.

GitHub Actions on every PR

All tests run automatically before merge.

Comprehensive Testing

Mocked Kafka, Flask, and telemetry systems.
Achieved **~85% test coverage**.

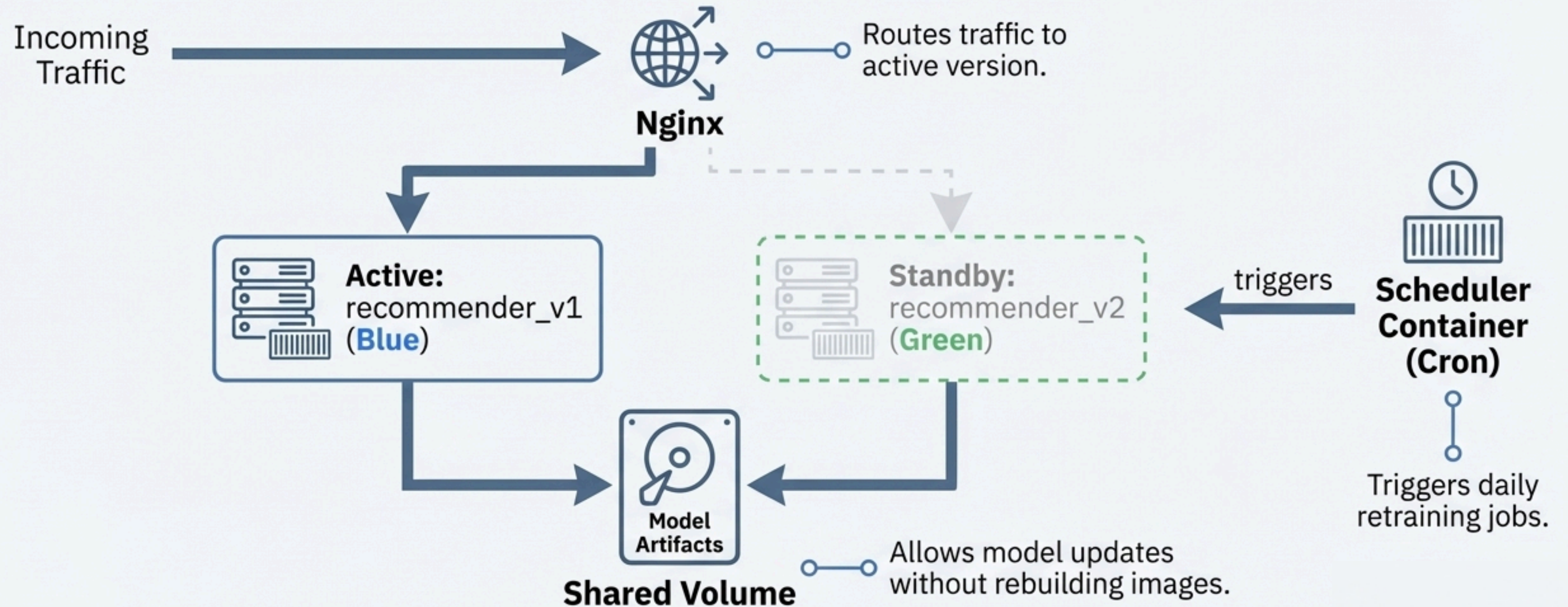


The Impact of Robust CI

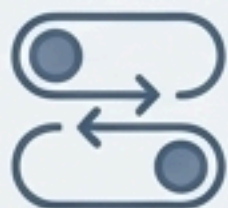
This wasn't just about catching bugs. It enforced a culture of production discipline, preventing silent breakages and ensuring that every change was deployable.

The Delivery: Deploying safely with zero-downtime rollouts

Our choice: A containerized blue/green architecture



Inside the Blue/Green Deployment Strategy.



Safe, Zero-Downtime Rollouts

- Switching versions is a simple Nginx configuration change.
- Allows for instant rollbacks if the new model underperforms.



Efficient Resource Management

- Shared mounted volume for models avoids rebuilding Docker images for every new model version.
- Multi-stage Dockerfile reduces final image size and improves cold-start time.



Clear Separation of Concerns

- Serving (Flask), Scheduling (Cron), and Load Balancing (Nginx) are managed in separate, independent containers.
- Mirrors industry best practices for microservice architecture.

The Control Tower: Ensuring full traceability and provenance.

Every prediction must be reproducible.

Automated Retraining & Versioning

A nightly Cron job produces:

- Timestamped SVD model artifact
- Updated top-20 fallback model
- A structured ``model_metadata.json``

Anatomy of ``model_metadata.json``

```
{
  "model_version": "2023-10-27T00:00:00Z",
  "git_sha": "a1b2c3d...",
  "dataset_fingerprint": {
    "size": "500MB",
    "rows": 1000000,
    "timestamp": "..."
  }
}
```

****Per-Request Provenance Logging****: The model version, route (cold/warm), and dataset ID are logged for every single inference call.

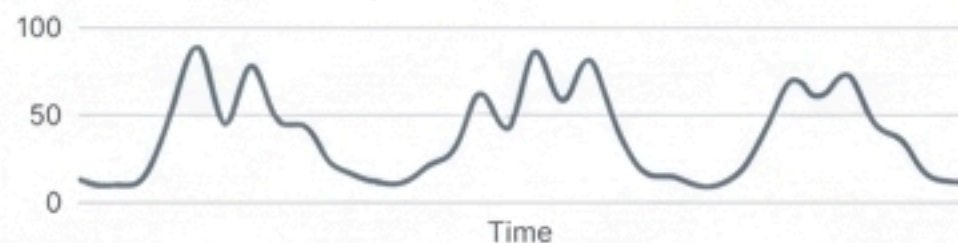
The Control Tower: Real-time visibility into system health and data drift

App Health

Latency (p95)



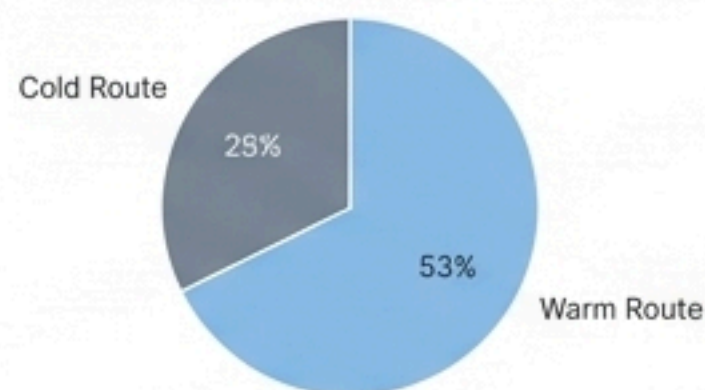
Requests Per Second



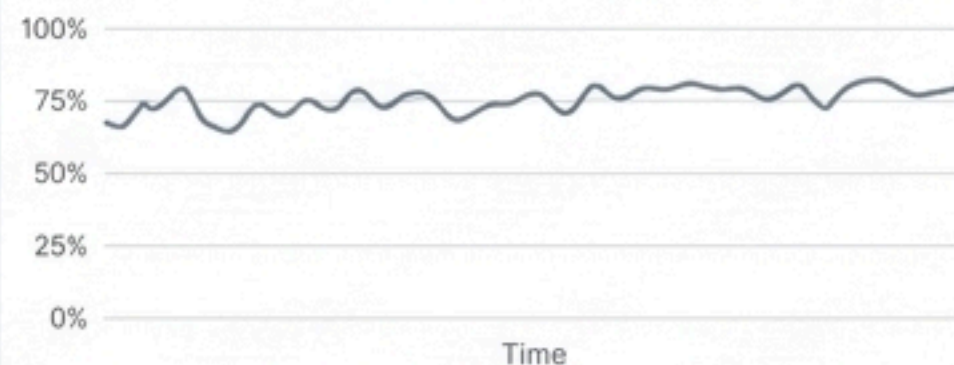
99.9%
Uptime

Telemetry

Cold vs. Warm Route

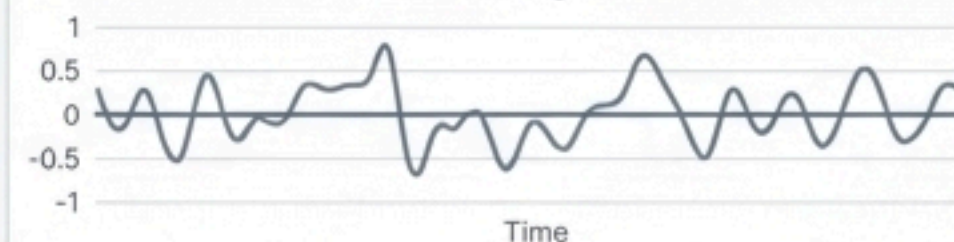


Hit Rate Over Time



Data Drift

Mean Rating Drift



KL Divergence



Rating Volume Changes



Automated Alerts: UI alerts are configured for critical events like service downtime or significant KL divergence.

Hit Rate By Route



Grafana Hit Rate Dashboard lets us monitor routing effectiveness

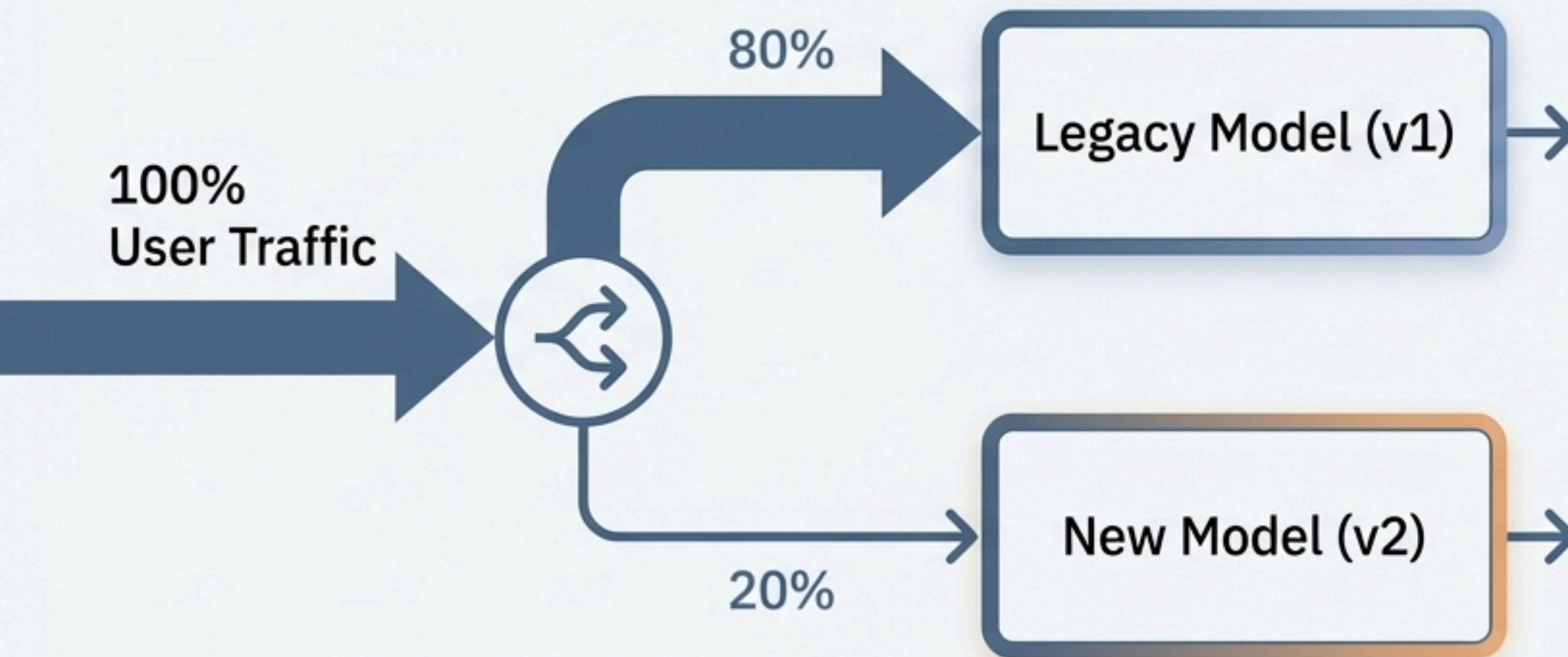
Request Latency Dashboard tracks if our system is meeting performance expectations

P95 Latency



The Control Tower: A framework for safe, continuous experimentation.

How do we safely validate a newly retrained model before full rollout?



Evaluation Process

1. Log model version usage in the telemetry pipeline.
2. Compute key business metrics (e.g., Hit Rate) per model version.
3. Run hourly t-tests to compare v1 vs. v2 performance for statistical significance.

While initial results were statistically insignificant, the framework is production-ready and enables continuous, safe model evaluation.

The Conscience: Investigating fairness for different user groups.

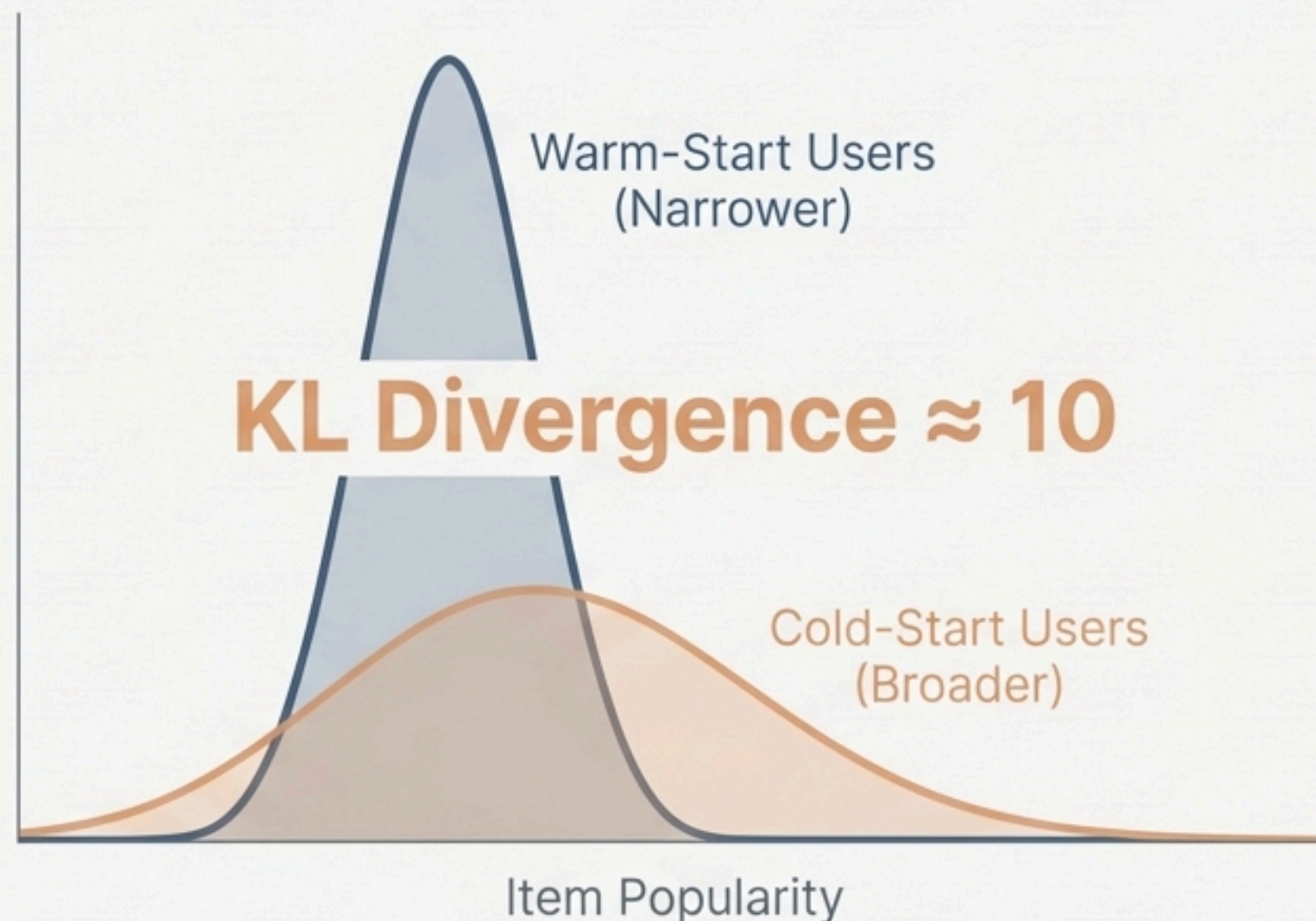
We analyzed two user cohorts: **Cold-Start** (new users) vs. **Warm-Start** (existing users).

Key Finding

Identified significant fairness concerns in recommendation exposure.

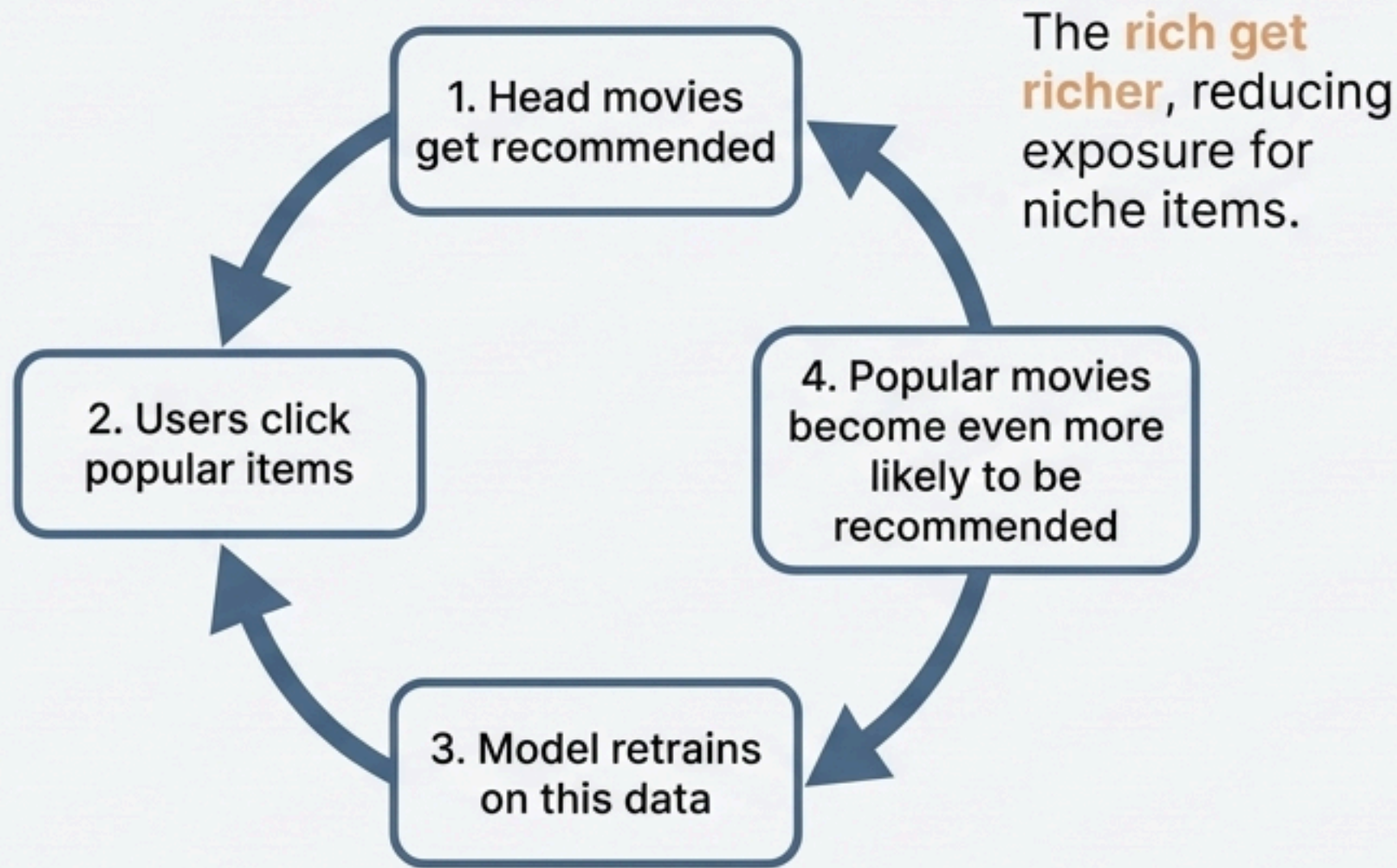
- Warm users receive a narrower, more personalized set of recommendations.
- There is a small but meaningful difference in hit rates between the two groups.

Exposure Distribution Divergence



The Conscience: Identifying the feedback loops driving unfairness.

Popularity Confirmation Loop



Cold-Start Identity Distortion

Cold-Start Identity Distortion



A new user's initial interactions are heavily influenced by the system's popular defaults. These early clicks then define their long-term user profile, potentially **trapping them in a bubble**.

The Conscience: Proposed mitigations to improve fairness

System-Level Interventions

- **Exploration Slots:** Reserve a portion of recommendation slots for less popular or novel items.
- **Stochastic Reranking:** Introduce randomness in the final ranking to break popularity cycles.
- **Fairness Dashboards:** Actively monitor exposure and hit rate divergence as a primary system metric.

Model-Level Improvements

- **Lightweight Content Signals:** Use genres/tags to provide initial recommendations for cold-start users instead of just global popularity.
- **Profile Smoothing:** When a user transitions from cold to warm, smooth their initial interaction data to reduce bias.

Key Lessons: It's about the interaction between ML, systems, and deployment.

- The biggest challenge was understanding how a decision in one area (e.g., model choice) impacts another (e.g., deployment and monitoring).
- Gained hands-on experience with production tools: **Docker**, **Nginx**, **Prometheus**, and **Grafana**.
- Learned to debug complex, multi-container workflows and noisy telemetry data.
- Developed a collaborative workflow for debugging issues across the entire stack—from data to infrastructure.
- Deepened our understanding of fairness, data bias, and **feedback loops** as first-class engineering problems.

Evolving the System: Next steps for production-level robustness

- **Enrich Features:** Integrate content features (genres, embeddings) earlier to fundamentally solve the cold-start problem.
- **Standardize MLOps Tooling:** Replace manual JSON provenance with a dedicated tool like **MLflow** or **DVC** for better experiment tracking.
- **Adopt Feature Stores & Model Registries:** Standardize pipelines and ensure consistency between training and serving environments.
- **Collect Richer Telemetry:** Move beyond clicks to capture signals like scroll depth and watch duration for more nuanced model evaluation.
- **Integrate Diversity-Aware Ranking:** Build fairness directly into the serving logic, not just as an offline analysis.

These improvements would elevate the system's robustness, fairness, and long-term maintainability.

The background features a light gray geometric pattern of thin lines and dots. The lines are arranged in a complex, overlapping manner, creating a sense of depth and structure. The dots are small and dark, serving as focal points for the lines. The overall aesthetic is modern and minimalist.

Thank you